

Project 2 Image Stitching

B10705012 李昊軒

B10705046 沈奕

Feature : Multi-Scale Oriented Patches

Detection : `harris_corner_detector(img, octave)`

Multi-Scale Harris Corner Detector

在`msop_DetectandDiscriptor(img)`中建立Gaussian pyramid，對每一層利用Gaussian Derivatives ($\sigma_d = 1.0$)得到 x 和 y 方向的梯度(在此使用近似作法，先用 Gaussian filter 再利用 `cv2.Sobel` 取出 gradient)，再計算 Harris Matrix，且計算每一格的corner strength f_{hm} 。

$$\text{Harris Matrix} = \nabla_{\sigma_d} P_1(x, y) \nabla_{\sigma_d} P_1(x, y)^T * g_{\sigma_i}(x, y) = \begin{bmatrix} I_{xx} & I_{xy} \\ I_{xy} & I_{yy} \end{bmatrix}, \sigma_i = 1.5$$

$$f_{hm} = \frac{\det(\text{Harris Matrix})}{\text{tr}(\text{Harris Matrix})}。$$

用`cv2.dilate`取出附近的最大值($\text{kernel} = 3 \times 3$)，並將`threshold`設成 100，保留 f_{hm} 大於`threshold`且為附近之最大值的點，記錄成`keypoints`與`scores`，`keypoints`為特徵點座標而`scores`為其corner strength。

Adaptive Non-Maximal Suppression

先根據輸入的`scores`(也就是 corner strength)進行遞減排序，獲得排序後的特徵點 `sorted_kps` 以及其對應的分數 `sorted_scores`。再初始化每個特徵點的半徑`radius`，初始值設為極大值(在此設為 `1e10`)，用來儲存每個點與其最近一個「比自己強的點」之間的距離`radius[i]`。接著從第二個特徵點開始，對於每一個點找出所有比它強的點，比自己強的點定義為

$$f_{hm}(x_j) * c_{robust} > f_{hm}(x_i)$$

並利用 KD tree計算與這些點中最近距離的點設為`radius[i]`。最後根據`radius`由大到小排序，選出前 `keypoint_num` (在此設為 500)個點作為最終保留的特徵點。

Orientation

同樣利用Gaussian Derivatives ($\sigma_o = 4.5$)得到 x 和 y 方向的梯度，且利用`cv2.magnitude`計算梯度長度`u_length`，並且計算

$$\cos\vartheta = \frac{\nabla P_x}{|u|}, \sin\vartheta = \frac{\nabla P_y}{|u|}$$

Descriptor : `extract_msop_descriptor(pyramid, keypoints)`

先建立 $(8 * spacing) \times (8 * spacing)$ 的 *meshgrid*，針對每個 *keypoint*，

利用他們的 $\cos\theta$ 、 $\sin\theta$ 旋轉 *meshgrid* 並加上 *keypoint* 座標，建立 *patch*。

$$\begin{bmatrix} x_{sample} \\ y_{sample} \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x_{grid} \\ y_{grid} \end{bmatrix} + \begin{bmatrix} x_{keypoint} \\ y_{keypoint} \end{bmatrix}$$

取得 40×40 的 *patch* 並 *reshape* 成 8×8 ，並將每個 *patch* 正規化成平均值為 0、標準差為 1，避免亮度差異。

$$normalized\ patch = \frac{patch - \overline{patch}}{\sigma_{patch}}$$

接下來在 *patch* 上用 *Haar wavelet transform* 去建立 64 維的 *descriptor* 並回傳。

Matcher : `knn_matcher(descriptor1, descriptor2)`

輸入兩張圖片的 *descriptors*，利用 *descriptor2* 建立 KD tree，並針對 *descriptor1* 中每個 *descriptor* 找其 *k* 個最近的 *descriptor* (在此 *k* 設為 2)，如果最近的 *descriptor* 距離小於 $ratio\ threshold * 距離第二近的\ descriptor$ 則加入 *matches*。(在此 *ratio threshold* 設為 0.8)

Image Matching

Stitching : `panorama(imgs)`

Canva Initialization

建立長度為最長照片長度兩倍，寬度為所有照片寬度總和的畫布，並將第一張圖放置於最左邊。

Feature Extraction

對每兩張連續的照片提取 MSOP 特徵，並轉換到 *homogeneous coordinates*。

RANSAC

利用 RANSAC 執行 *k* 次 (在此 $K = 2000$)，每次隨機挑選 *n* 個點 (在此 $N = 4$)，並利用 *solve_homography* 計算出 Homography *H*。
將第二章圖的特徵點利用 *H* 轉換到新座標並計算 *inlier* 數 (在此預設為新座

標與其 match 的點在 3 個 pixels 內即算 inlier)，最終紀錄 inlier 數最多的 H 為最好的 Homography best_H。

Chain Homography

因 best_H 為第二張圖與第一張圖轉換的 Homography，因此需要轉換到最終結果 dst 的 Homography，因此最終的 Homography 為 last_best_H。

$$\text{last best H} = \text{last best H} \cdot \text{best H}$$

Homography : solve_homography(u, v)

為了計算 H 矩陣， $H = \begin{bmatrix} h_{11} & h_{12} & h_{31} \\ h_{21} & h_{22} & h_{32} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$ ，我們利用下列公式計算。

$$v_x = \frac{h_{11}u_x + h_{12}u_y + h_{13}}{h_{31}u_x + h_{32}u_y + h_{33}}$$

$$v_y = \frac{h_{21}u_x + h_{22}u_y + h_{23}}{h_{31}u_x + h_{32}u_y + h_{33}}$$

因此可得出

$$A = \begin{bmatrix} u_{x,1} & u_{y,1} & 1 & 0 & 0 & 0 & -u_{x,1}v_{x,1} & -u_{y,1}v_{x,1} & -v_{x,1} \\ 0 & 0 & 0 & u_{x,1} & u_{y,1} & 1 & -u_{x,1}v_{y,1} & -u_{y,1}v_{y,1} & -v_{y,1} \\ u_{x,2} & u_{y,2} & 1 & 0 & 0 & 0 & -u_{x,2}v_{x,2} & -u_{y,2}v_{x,2} & -v_{x,2} \\ 0 & 0 & 0 & u_{x,2} & u_{y,2} & 1 & -u_{x,2}v_{y,2} & -u_{y,2}v_{y,2} & -v_{y,2} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix},$$

使得 $Ah = v$ ，接著對 A 做奇異值分解，用 V 的最後一個 column 當作 H。

Warping : warping(src, dst, H, ymin, ymax, xmin, xmax)

將 src warp 到 dst，其中 warping function 使用的是 backward warping 避免出現 hole，且對於非整數點採用 nearest neighbor 的值，其中 mask 為 backward warping 中計算出 src 座標後檢查是否超出 src 範圍。

Blending : Alpha Blending

Blender : `blend_two_images(im1, im2)`

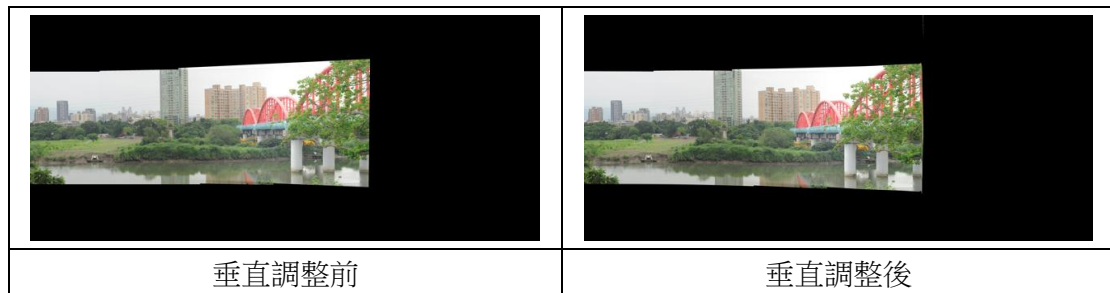
將兩張影像在 `warping` 時平滑融合成一色調一致之影像，以避免在重疊區域產生明顯接縫。此處用的是 `alpha blending` 線性方式混合，混合範圍為兩張照片重疊接縫處，而 `alpha` 本身為決定兩張照片重疊處 `color channel` 之權重，根據混合處之 `x` 座標靠近左圖邊界抑或是右圖，越靠近該圖則該圖權重越高，為線性升降。



Adjustment

drift : `correct_vertical_drift(panorama)`

修正因相機擺動導致之垂直偏移。透過找到圖片 `y` 座標的中心點建立輪廓線並且平滑化、再找出平滑後的 `boundary_y` 中位數作為 `reference line`，據其差距進行垂直調整。

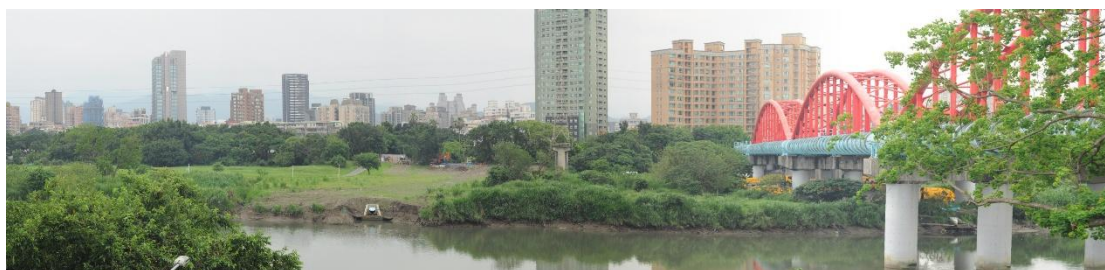


cropping : find_max_rectangle(mask)

將得到剛處理好、原為不規則形的全景圖，裁切出最大的矩形。先建立對每一列，統計該像素以上連續非零像素的個數，使用遞增 **stack** 來維護每一列高度的 **index**，並找出最大內切矩形進行裁切。



Result



Experiment and Comparison

一開始在 **feature detection** 時，我們的程式最主要的時間瓶頸在 **ANMS**，當一張越複雜圖輸入，一開始 **threshold** 設為 10，**ANMS** 會耗費大量時間，因此必須根據不同的圖片調整 **threshold**。除此之外，在 **Gaussian Pyramid** 中我們發現，金字塔在 1 層、2 層級 4 層皆可以完美拼接圖片，因此在 **Panorama** 應用中，**Multi-Scale** 似乎不是那麼重要。最後，我們也有嘗試一開始投影到援助座標上，但結果出來最後一張圖片似乎會被過度拉伸，推測是因為我們的 **Homography** 不是單純的 **translation** 矩陣。